Fixed-Point FIR Filter and Quantized Neural Network Accelerator

RTL Design, Verification, and Synthesis

Abstract

This project is presented in two linked phases. Phase 1 designs, implements, and verifies an 8-tap fixed-point FIR low-pass filter in two RTL architectures — a fully parallel direct-form design and a resource-shared serial design — and compares them after logic synthesis in terms of gate count, flip-flop count, and throughput. Phase 2 extends this foundation into a small hardware-aware quantization study: a tiny fully-connected neural network is trained on MNIST in PyTorch, quantized to fixed-point at several bit-widths, and the 8-bit version is re-implemented as a synthesizable RTL inference accelerator built from the same multiply-accumulate (MAC) primitive used in the FIR filter. Both designs are functionally verified against Python reference models using self-checking testbenches, and both are synthesized using the open-source Yosys toolchain to obtain real hardware resource figures.

Contents

1. Introduction and Motivation

2. Toolchain

3. Phase 1 – FIR Filter Design

3.1 Direct-Form Architecture

3.2 Serial (Resource-Shared) Architecture

3.3 Verification Methodology

3.4 Synthesis Results

4. Phase 2 – Neural Network Inference Accelerator

4.1 Model and Training

4.2 Quantization Study

4.3 RTL Architecture

4.4 Verification Against Python Reference

4.5 Synthesis Results

5. Comparative Discussion

6. Limitations and Future Work

7. Conclusion

1. Introduction

The goal of this project was to build, from first principles, a small but complete and honestly verified body of digital design work spanning RTL design, functional verification, and logic synthesis — the three stages that make up the front end of any real VLSI design flow. An FIR (Finite Impulse Response) filter was chosen as the starting point because it is a canonical signal-processing building block whose correctness is easy to check (its impulse response is literally its coefficient set) while still requiring genuine sequential logic, pipelining awareness, and testbench discipline to implement correctly. The project was then deliberately extended beyond a single filter exercise into a small, original hardware-aware quantization study of a neural network accelerator, since the multiply-accumulate datapath used throughout both phases is the same fundamental building block as the filter's resource-shared design.

2. Toolchain

Stage	Tool	License
RTL design	Verilog-2005 (hand-written)	N/A
Simulation	Icarus Verilog	Free, open-source
Waveform debugging	GTKWave	Free, open-source
Logic synthesis	Yosys (generic cell library flow)	Free, open-source
Golden reference models	Python (NumPy, PyTorch)	Free, open-source
Model training data	MNIST (via torchvision)	Public dataset

3. Phase 1 – FIR Filter Design

An 8-tap FIR low-pass filter was implemented twice, using two different hardware architectures that trade area for throughput in opposite directions. Both operate on 8-bit signed samples and 8-bit signed coefficients and produce an 18-bit accumulated output.

3.1 Direct-Form Architecture

The direct-form design (rtl/fir_direct.v) uses eight parallel multipliers and an 8-input adder tree, all operating every clock cycle. An 8-deep shift register holds the eight most recent input samples. This produces one filtered output sample per clock cycle, at the cost of eight simultaneous multiply-accumulate units in hardware.

3.2 Serial (Resource-Shared) Architecture

The serial design (rtl/fir_serial.v) uses a single shared multiplier and accumulator, reused across all eight taps via a small sequencing counter and a busy/sample_valid/y_valid handshake. One output sample now takes eight clock cycles to produce, but only one multiplier is instantiated in hardware instead of eight.

3.3 Verification Methodology

A Python golden reference model (sim/fir_reference.py) computed the expected output for 20 pseudo-random 8-bit signed input samples using NumPy convolution against the same coefficient set used in the RTL. Two self-checking testbenches (tb/fir_direct_check_tb.v, tb/fir_serial_check_tb.v) read these vectors from a text file, drove them into the respective design, and automatically compared the

RTL output against the Python-computed expected value — accounting for the measured pipeline latency of each architecture (1 cycle for direct-form; an 8-cycle handshake-gated latency for serial). Both testbenches reported **ALL TESTS PASSED** across the full 20-vector set.

A notable debugging step during this phase was a race condition in the initial direct-form testbench, where stimulus changes applied exactly at a clock edge raced against the design's own posedge clk sampling, producing an off-by-one-sample impulse response. This was resolved by inserting a small delay (#1) after each clock edge before changing stimulus — standard, necessary testbench practice.

3.4 Synthesis Results

Both designs were synthesized using Yosys's generic synthesis flow (technology-independent cell mapping, i.e. not targeting a specific FPGA or ASIC library), which gives directly comparable gate-level cell counts.

Metric	Direct-form	Serial	Observation
Total cells	926	880	Comparable overall size
Flip-flops	64	105	Serial needs extra control state
AND gates	409	341	Serial lower
OR gates	192	193	~Equal
XOR gates	261	203	Serial lower
MUX / NOT (control)	0 / 0	17 / 21	Serial adds sequencing logic
Throughput	1 sample / cycle	1 sample / 8 cycles	8x throughput difference

***Key finding*:** The expected area benefit of resource sharing did not appear strongly at this scale. Flip-flop count actually increased for the serial design (64 → 105), because the control overhead needed to share one multiplier — a tap counter, an accumulator register, and a busy flag — is comparable in size to the seven multipliers it removes, at only 8 taps and 8-bit precision. Combinational logic (AND/XOR) did drop, but modestly. The conclusion drawn is that resource-sharing's area payoff depends on the multiplier being expensive relative to its control overhead; it would be expected to become clearly favourable with wider data (16/24-bit) or more taps (16, 32), where the saved multiplier cost grows faster than the fixed control-logic cost. This is noted as a natural direction for future work. Separately, the throughput difference (1 cycle vs. 8 cycles per output) is real and architecture-defining regardless of the gate-count result.

4. Phase 2 – Neural Network Inference Accelerator

Phase 2 extends the same multiply-accumulate design pattern used in the FIR filter's serial architecture into a small, genuinely research-flavoured question: how does the bit-width used to represent a neural network's weights and activations trade off against classification accuracy, and what does that network cost to implement in actual digital hardware?

4.1 Model and Training

A small fully-connected network (784 → 32 → 10, i.e. flattened MNIST pixels into a single hidden layer of 32 ReLU neurons, into 10 output logits) was trained from scratch in PyTorch for 5 epochs using Adam and cross-entropy loss (ml/train_mlp.py). The trained float32 model achieved **93.72% test accuracy** on the MNIST

test set, which forms the baseline against which every quantized version is compared.

4.2 Quantization Study

Symmetric per-tensor quantization was implemented in Python and applied at 16, 8, 6, and 4-bit precision, first to weights only (ml/quantize.py), then to both weights and activations (ml/quantize_full.py) — the latter is the accurate predictor of what dedicated fixed-point hardware actually computes, since real hardware cannot carry full-precision floating-point values between layers.

Bit-width	Weights only	Weights + Activations	Drop from FP32
16-bit	93.72%	93.72%	0.00 pts
8-bit	93.64%	93.63%	0.09 pts
6-bit	93.50%	93.42%	0.30 pts
4-bit	90.51%	88.97%	4.75 pts

***Key finding*:** Accuracy is essentially unaffected down to 8-bit precision and still reasonable at 6-bit, with a clear accuracy "cliff" appearing only at 4-bit, where compounding weight-and-activation quantization error costs nearly 5 accuracy points. This directly motivated the choice of **8-bit** as the precision implemented in RTL — it preserves accuracy within 0.1 points of the float32 baseline while halving the bit-width (and thus the hardware cost) relative to 16-bit.

4.3 RTL Architecture

A single parameterized module, mac_layer (rtl/mac_layer.v), implements one fully-connected layer: it accepts IN_SIZE and OUT_SIZE as compile-time parameters, loads int8 weights from a hex file via $readmemh, and sequences through one multiply-accumulate per clock cycle per output neuron, optionally applying a ReLU non-linearity on completion (controlled by an APPLY_RELU parameter). This is architecturally the same resource-shared pattern as the FIR filter's serial design, scaled up and generalized.

The top-level module accelerator_top.v instantiates this module twice — once parameterized as 784 → 32 with ReLU for the hidden layer, once as 32 → 10 without ReLU for the output layer — and sequences them with a small finite-state machine that feeds layer 1's output directly into layer 2's input.

***Inter-layer rescaling*:** fixed-point accumulators grow in magnitude as more terms are summed, so a layer's raw 32-bit accumulated output cannot be fed directly into the next 8-bit layer without rescaling. An initial naive attempt that simply truncated the accumulator's low 8 bits was found, by direct testbench debugging, to discard essentially meaningful information and produced an incorrect prediction. This was corrected by implementing a calibrated right-shift-and-saturate rescale (dividing by a power of two and clamping to the int8 range), a standard, low-cost technique used in real low-power accelerator designs to avoid a full fixed-point multiplier between layers.

4.4 Verification Against Python Reference

Ten real MNIST test images were quantized to int8 in Python, exported as hex files, and driven through the complete RTL accelerator (tb/accelerator_tb.v), with the predicted digit (the index of the largest output logit) compared against the ground-truth label for each image.

Image	True label	RTL prediction	Match
0	7	7	Yes
1	2	2	Yes
2	1	1	Yes
3	0	0	Yes
4	4	4	Yes
5	1	1	Yes
6	4	4	Yes
7	9	9	Yes
8	5	6	No
9	9	9	Yes

***Result*: 9/10 correct.** The single mismatch (image 8, true label 5, predicted 6) is the same image that Python's own int8-quantized model misclassifies — i.e. the RTL implementation reproduces the quantized software model's behaviour exactly, including its one genuine error. This is interpreted as strong evidence of implementation correctness: the RTL is faithful to the quantized model it was derived from, rather than introducing independent hardware bugs.

4.5 Synthesis Results

The complete accelerator was synthesized with the same generic Yosys flow used for the FIR filters.

Metric	Count
Total cells	102,435
Multiplexers ($_MUX_)	90,934 (~89% of all cells)
AND gates	3,011
XOR gates	553
OR gates	754
Flip-flops (all types)	~7,050

***Key finding*:** Resource usage is overwhelmingly dominated by multiplexers, not arithmetic logic. This is a direct structural consequence of implementing the weight memory (25,088 values for layer 1 alone) as flip-flop-based storage with a runtime-computed address (neuron_idx * IN_SIZE + tap_idx): without inferred block RAM, Yosys's generic flow must synthesize a large select-1-of-N multiplexer tree for every memory read. On a real FPGA target, a vendor-specific synthesis flow (e.g. Xilinx Vivado) would typically infer this as dedicated block RAM (BRAM) resources, reducing this cost substantially — this is explicitly flagged as a limitation of the generic flow used here, not a property of the underlying algorithm.

5. Comparative Discussion

Design	Total cells	Dominant resource	Bottleneck type
FIR direct-form	926	AND/OR/XOR	Arithmetic-bound
FIR serial	880	AND/OR/XOR (+control)	Arithmetic-bound
NN accelerator	102,435	MUX (89%)	Memory-addressing-bound

The two phases of this project expose qualitatively different hardware bottlenecks for a similar underlying operation (multiply-accumulate). The FIR filter, with its small, fixed 8-element working set, is dominated by arithmetic logic. The neural network accelerator, despite reusing the exact same MAC pattern, is dominated by memory-addressing logic because its working set (25,088 weight values) is far too large to address with fixed wiring. This is itself a useful, transferable conclusion: workload size, not just operation type, determines where hardware cost actually concentrates, and this has direct implications for which optimizations (e.g. BRAM inference, weight reuse scheduling) would matter most for each design.

6. Limitations and Future Work

- Generic (non-vendor-specific) synthesis was used throughout; a Vivado or OpenROAD flow targeting a real FPGA/ASIC library, with explicit block-RAM inference for the accelerator's weight memory, would give more representative area and frequency numbers.
- Maximum clock frequency (Fmax) was not measured, as this requires a place-and-route step beyond generic synthesis; this is scoped out and left as future work rather than approximated.
- The accelerator's inter-layer rescaling uses a single calibrated shift amount rather than a properly derived per-layer fixed-point multiply; this is a deliberate scope simplification, documented at the point in the RTL where it applies.
- Only weight-and-activation quantization was studied; per-channel (rather than per-tensor) quantization, which typically recovers additional accuracy at low bit-widths, was not explored.
- The FIR area comparison was performed at a fixed scale (8 taps, 8-bit data); the hypothesis that resource-sharing's area benefit improves at larger scale (wider data or more taps) was reasoned about but not experimentally verified within this project's timeline.

7. Conclusion

This project delivered two fully verified, synthesized RTL designs — an FIR filter studied across two architectures, and a quantized neural network accelerator built from the same underlying MAC primitive — along with an original, data-backed quantization study connecting software-level accuracy results to hardware-level resource costs. Both designs were verified against independent Python golden models using automated, self-checking testbenches rather than manual inspection, and both were carried through to real logic synthesis rather than stopping at simulation. The honest documentation of debugging steps (the testbench race condition in Phase 1; the rescaling bug in Phase 2) and of scope limitations (generic synthesis, no Fmax, no BRAM inference) reflects the actual, complete working process of the project.